Programación orientada a objetos

1. Características

La Programación Orientada a Objetos (POO) viene a cambiar la forma de programar. Los objetos se utilizan como metáfora para emular las entidades reales del negocio a modelar.

Está basada en varias técnicas como: abstracción, polimorfismo, encapsulamiento, herencia y cohesión/acoplamiento.

1. **Abstracción**: La abstracción implica enfocarse en las características esenciales de un objeto, ignorando las menos importantes o accidentales. En POO, esto se traduce en la creación de clases que representan entidades abstractas con atributos y métodos relevantes. Ejemplo en Kotlin:

```
abstract class Vehiculo(val marca: String, val modelo: String) {
    abstract fun acelerar()
    abstract fun frenar()
}
```

Aquí, `Vehiculo` es una clase abstracta que define la abstracción de un vehículo con características comunes como marca y modelo, y acciones como acelerar y frenar.

Las clases abstractas no pueden ser instanciadas directamente; ***se utilizan como base para otras clases***.

1. **Polimorfismo**: El polimorfismo permite que objetos de diferentes clases sean tratados como objetos de una clase común. Esto se logra a través de la herencia y la implementación de interfaces. Ejemplo en Kotlin:

```
class Coche(marca: String, modelo: String) : Vehiculo(marca, modelo) {
    override fun acelerar() {
        println("Coche acelerando")
    }

    override fun frenar() {
        println("Coche frenando")
    }
}
```

`Coche` es una clase que hereda de `Vehiculo` y proporciona su propia implementación de los métodos acelerar y frenar.

1. **Encapsulamiento**: El encapsulamiento implica restringir el acceso directo a algunos componentes de un objeto y proteger la integridad del objeto exponiendo solo lo que es necesario. Ejemplo en Kotlin:

```
class CuentaBancaria(private var saldo: Double) {
    fun depositar(monto: Double) {
        if (monto > 0) {
            saldo += monto
        }
    }
```

```
    fun consultarSaldo(): Double {
        return saldo
    }
}
```

Aquí, el saldo de `CuentaBancaria` está encapsulado. Se manipula únicamente a través de métodos como `depositar`.

1. **Herencia**: La herencia es un mecanismo mediante el cual una nueva clase adquiere las propiedades y comportamientos de otra clase. Esto ayuda en la reutilización de código. Ejemplo en Kotlin:

```
open class Animal(val nombre: String) {
    open fun sonido() {
        println("El animal hace un sonido")
    }
}

class Perro(nombre: String) : Animal(nombre) {
    override fun sonido() {
        println("El perro ladra")
    }
}
```

`Perro` hereda de `Animal` y sobrescribe el método `sonido`.

1. **Cohesión/Acoplamiento**: La cohesión se refiere a qué tan estrechamente relacionadas están las tareas realizadas por una clase. El acoplamiento se refiere a cómo de interdependientes son las clases entre sí. La alta cohesión y el bajo acoplamiento son ideales en POO para facilitar la mantenibilidad y la escalabilidad.
2. **Alta cohesión**: Una clase `Calculadora` que realiza operaciones matemáticas.
3. **Bajo acoplamiento**: `Calculadora` utiliza una interfaz `OperacionesMatematicas` para interactuar con otras clases, lo que reduce su dependencia directa.

Estos conceptos son fundamentales en POO y ayudan a crear software bien estructurado, mantenible y escalable.

2. Clases y objetos/instancias

En la programación orientada a objetos (POO), los conceptos fundamentales son las clases y los objetos. Estos conceptos son esenciales para entender cómo la POO modela el mundo real y soluciona problemas de forma eficiente y organizada.

2.1 Conceptos

- Las **clases** son una plantilla para la creación de objetos de datos según un modelo predefinido.
- Los **objetos** son una instancia de la clase, es decir una concreción. Es un ejemplo de esa clase.

Ej:

- Clases: Persona, Animal, Coche
- Objetos/Instancias: Juan, Perro, Seat Ibiza

Las clases tienen los siguientes componentes:

- **Atributos/campos**: también llamadas variables de instancia, contienen la información de los futuros objetos. Normalmente privadas al objeto. Los atributos son otros objetos que cualifican al objeto y se acceden a través del . y el nombre del atributo.
- **Métodos**: tambien llamados métodos de instancia, implementan la funcionalidad o comportamiento asociado al objeto, son funciones. Se acceden a estors métodos a través del . y el nombre del método. Estos métodos pueden acceder a las variables de instancia de la clase directamente. Cuando se desea realizar una acción sobre un objeto, se dice que se le manda un **mensaje** invocando a un método que realizará la acción.
- **Constructores**: son métodos especiales que se utilizan para inicializar los objetos. Se llaman automáticamente cuando se crea un objeto de la clase. Además van acompañados de un bloque de inicialización.

Los objetos tienen los siguientes componentes:

- **Identidad**: Los identifica del resto de objetos de la misma clase.
- **Comportamiento**: Relacionado con su funcionalidad, determina las operaciones que se pueden realizar y con las que responde a los mensajes.
- **Estado**: Conjunto de atributos y sus valores en un momento determinado.

2.2 Creando clase

2.2.1 Constructores

Un **constructor** es un método especial que se utiliza para inicializar los objetos. Se llama automáticamente cuando se crea un objeto de la clase.

En Kotlin, una clase puede tener tanto un constructor primario como uno o más constructores secundarios. Si la clase tiene un constructor primario, el secundario debe delegar en el constructor primario, ya sea directamente o indirectamente usando la palabra clave `this`. Todos los constructores son públicos por defecto, lo que significa que son visibles donde sea visible la clase.

2.2.1.1 Constructor Primario

Se declara en el encabezado de la clase y es el más comúnmente utilizado. Por ejemplo:

```
class Persona(val nombre: String, val edad: Int)
```

Aquí, `Persona` tiene un constructor primario con dos parámetros: `nombre` y `edad`. Si no hay anotaciones ni modificadores de visibilidad, la palabra clave `constructor` es opcional y se puede omitir.

2.2.1.2 Constructor Secundario

Se declara dentro del cuerpo de la clase y se utiliza para proporcionar funcionalidades adicionales o para permitir diferentes formas de instanciar un objeto.

```
class Persona(val nombre: String) {
  var edad: Int = 0

  constructor(nombre: String, edad: Int) : this(nombre) {
      this.edad = edad
  }
}
```

En el ejemplo anterior, `Persona` tiene un constructor primario que solo toma un `nombre`, y un constructor secundario que toma tanto un `nombre` como una `edad`. Presta atención a como el constructor secundario delega en el constructor primario usando la palabra clave `this`.

2.2.1.3 Constructor vacío

Si no se proporciona ningún constructor, Kotlin crea un constructor vacío por defecto. Por ejemplo:

```
  class Persona
```

2.2.2 Definición de las variables de instancia en el constructor

Por otra parte, las propiedades o variables de instancia pueden definirse en el constructor principal o en el cuerpo de la clase. En el ejemplo anterior, `nombre` es una propiedad definida en el constructor principal, mientras que `edad` es una propiedad definida en el cuerpo de la clase.

2.2.2.1 Definición en el Constructor

Al usar `val` o `var` en el constructor principal, Kotlin genera automáticamente métodos getter y setter.

- `val`: Solo se crea un método getter, ya que `val` define una propiedad de solo lectura.
- `var`: Se crean tanto getter como setter, permitiendo lectura y escritura.

Ejemplo:

```
class Persona(val nombre: String, var edad: Int)
```

2.2.2.2 Definición Personalizada en el Cuerpo de la Clase

Puedes personalizar cómo se accede y modifica una propiedad mediante la definición explícita de los métodos getter y setter.

Ejemplo:

```
class Persona(nombreInicial: String, edadInicial: Int) {
var nombre: String = nombreInicial
    get() = field.toUpperCase() // Personaliza el getter
    set(value) {
        field = value.capitalize()
    }

var edad: Int = edadInicial
    private set // Solo getter público, setter privado
}
```

Aquí, `nombre` tiene un getter personalizado que devuelve el nombre en mayúsculas y un setter que capitaliza el valor asignado. `edad` tiene un setter privado, lo que significa que solo puede modificarse dentro de la clase `Persona`.

2.2.3 Bloques de inicialización

Los bloques de inicialización complementan los constructores primarios. Se utilizan para ejecutar código de inicialización que se requiere para todas las instancias de la clase, independientemente del constructor utilizado.

Los **Bloques de Inicialización `init`** se utilizan junto con el constructor primario. El código en estos bloques se ejecuta cada vez que se crea una instancia de la clase, y puede haber más de uno.

Dentro de los bloques `init` puedes usar la palabra `required` para validar las propiedades o parámetros. También puedes usar los parámetros del constructor primario en los bloques `init` y en los inicializadores de las propiedades en el cuerpo de la clase:

```
class Persona(val nombre: String, val edad: Int) {
  val nombreEnMayusculas = nombre.toUpperCase()
  init {
      require(nombre.trim().length > 0) { "Argumento nombre inválido."
}
      require(edad >= 0 && edad < 150) { "Argumento edad inválido." }
      println("Se ha creado una nueva Persona llamada $nombre")
  }
}
```

Una buena práctica es conocer el orden de ejecución de cada bloque.

```
class InitOrderDemo(name: String) {
    val firstProperty = "First property: $name"
    init {
        println("First initializer block that prints ${name}")
    }

    val secondProperty = "Second property: ${name.length}"
    init{
        println("Second initializer block that prints ${name.length}")
    }
}
```

2.3 Atributos vs propiedades

Mientras que un atributo es simplemente una variable dentro de una clase, una propiedad en Kotlin es una interfaz más sofisticada para acceder y modificar estos atributos, ofreciendo control adicional y encapsulamiento.

- **Atributos**: Son las variables definidas dentro de una clase. Por ejemplo, en una clase `Coche`, un atributo podría ser `marca`:

```
class Coche {
    private var marca: String = "Toyota" // Atributo, con acceso tanto
en lectura como en escritura privada.
}
```

Aquí, `marca` es un atributo de la clase `Coche`. Es privado, lo que significa que solo puede accederse dentro de la clase `Coche`.

- **Propiedades**: En Kotlin, una propiedad combina un atributo con sus métodos getter y setter. Estos métodos controlan el acceso y la actualización del atributo. Por ejemplo:

```
class Coche {
    var marca: String = "Toyota"
        private set // Solo se puede consultar. El setter es privado

    val info: String
        get() = "Coche de marca $marca" // Getter personalizado,
muestra una cadena con la marca.
}
```

Aquí, `marca` es una propiedad. Tiene un setter privado, lo que significa que no puedes cambiar `marca` desde fuera de la clase `Coche`. `info` es una propiedad de solo lectura con un getter personalizado que devuelve una cadena que incluye el valor de `marca`.

2.4 Ejemplos de clases y objetos personalizados

Vamos a crear ejemplos para las clases `Persona`, `Animal`, y `Coche`, así como para sus objetos o instancias específicas. Cada clase incluirá atributos (o campos) y métodos, y mostraré cómo se crean y utilizan las instancias de estas clases.

Clase `Persona`:
Atributos: nombre, edad
Métodos: presentarse

```
class Persona(private val nombre: String, private val edad: Int) {
    fun presentarse() {
        println("Hola, mi nombre es $nombre y tengo $edad años.")
    }
}

// Creando una instancia
val juan = Persona("Juan", 30)
juan.presentarse() // "Hola, mi nombre es Juan y tengo 30 años."
```

ten en cuenta que es lo mismo que lo siguiente:

```
//Definiendo la clase
class Persona(nombreInicial: String, edadInicial: Int) {
private var nombre: String = nombreInicial
private var edad: Int = edadInicial

    fun presentarse() {
        println("Hola, mi nombre es $nombre y tengo $edad
años.")
    }
}

// Uso de la clase para crear una instancia
val juan = Persona("Juan", 30)
juan.presentarse() // "Hola, mi nombre es Juan y tengo 30 años."
```

Clase `Animal`:
Atributos: especie, nombre
Métodos: emitirSonido

```
//Definiendo la clase
class Animal(private val especie: String, private val nombre: String)
{
    fun emitirSonido() {
        println("Soy un $especie y me llamo $nombre. ¡Hago un sonido
característico!")
    }
}

// Uso de la clase para crear una instancia
val perro = Animal("perro", "Firulais")
perro.emitirSonido() // Soy un perro y me llamo Firulais. ¡Hago un
sonido característico!
```

1. **Clase `Coche`**:
2. Atributos: marca, modelo
3. Métodos: describir

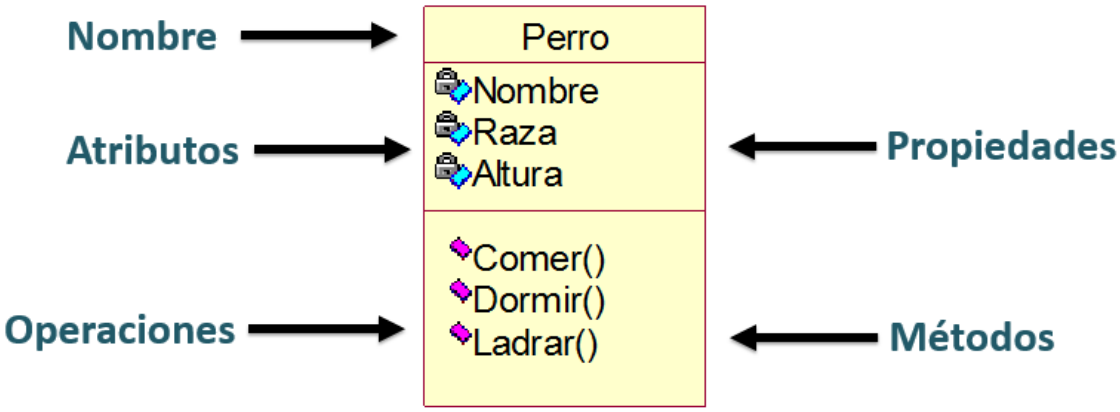
```
//Definiendo la clase
class Coche(private val marca: String, private val modelo: String) {
    fun describir(prefijo:String  = "") {
       println(prefijo+"Soy un coche de la marca $marca y modelo
$modelo.")
    }
}

// Uso de la clase para crear una instancia
val seatIbiza = Coche("Seat", "Ibiza")
seatIbiza.describir() // Soy un coche de la marca Seat y modelo Ibiza.
```

En cada una de estas clases, los atributos son privados, encapsulando así la información dentro de la clase. Los métodos `presentarse`, `emitirSonido`, y `describir` permiten interactuar con estas clases, mostrando cómo los objetos (instancias) pueden usar sus métodos para realizar acciones específicas.

Crear instancias como `juan`, `perro`, y `seatIbiza` de sus respectivas clases demuestra cómo las clases sirven como plantillas para crear objetos específicos con estados y comportamientos definidos.

Presta atención al `private` delante de los atributos de las clases. Esto es una buena práctica para encapsular los atributos y que no puedan ser modificados desde fuera de la clase.

Un ejemplo de representación de una clase `Perro` con tres atributos y 3 métodos.



2.5 Atributos y métodos estáticos (companion object)

Los métodos y atributos estáticos son elementos de una clase que pertenecen a la clase misma, en lugar de a instancias individuales de esa clase. En Kotlin, estos se manejan principalmente a través de `companion objects`. Las características clave incluyen:

1. **Atributos Estáticos**: Son variables asociadas con la clase en su conjunto. Por ejemplo, un contador de cuántas instancias de una clase se ha creado.
2. **Métodos Estáticos**: Son funciones asociadas con la clase, no con instancias individuales. Puedes llamar a estos métodos sin crear un objeto de la clase.

Los `companion objects` en Kotlin proporcionan una manera elegante de trabajar con propiedades y métodos que son comunes a todas las instancias de una clase, o que

deben ser accesibles sin necesidad de crear una instancia de dicha clase, además pueden usarse para otras funciones importantes:

1. **Almacenar Propiedades y Métodos Estáticos**: A diferencia de Java, Kotlin no tiene campos o métodos estáticos. Los `companion objects` proporcionan una funcionalidad similar, permitiendo definir propiedades y métodos que pertenecen a la clase en sí, y no a una instancia específica.
2. **Inicialización de Clase**: Pueden usarse para ejecutar código que prepare recursos necesarios para la clase antes de que se creen instancias.
3. **Patrones de Diseño**: Son útiles en la implementación de patrones de diseño como Singleton, Factory, entre otros.
4. **Interfaces**: Un `companion object` puede implementar interfaces, permitiendo que una clase tenga métodos estáticos que forman parte de una interfaz.

Un buen ejemplo para demostrar un `companion object` en Kotlin, usando una de las clases anteriores, podría ser en la clase `Animal`. Imagina que queremos llevar un registro del número total de animales creados. Esto es un dato que no pertenece a un solo animal, sino a la clase `Animal` en su conjunto. Podemos usar un `companion object` para esto:

```
class Animal(private val especie: String, private val nombre: String)
{
    companion object {
        private var contador = 0

        fun getTotalAnimales(): Int {
            return contador
        }
    }

    init {
        contador++
    }

    fun emitirSonido() {
        println("Soy un $especie y me llamo $nombre. ¡Hago un sonido
característico!")
    }
}
```

En este ejemplo, cada vez que se crea un nuevo `Animal`, el contador en el `companion object` se incrementa. Esto nos permite llevar un seguimiento de cuántos objetos `Animal` se han creado. El método `getTotalAnimales` es estático y se puede llamar sin una instancia de `Animal`, lo que es útil para obtener información a nivel de clase.

También, como ejemplo del uso de `companion object`, podríamos añadir a la clase `Coche` que defina si los coches tienen el volante a la derecha o a la izquierda. Esto puede depender de un valor estático como el país para el cual se crea el coche. Aquí te muestro cómo:

```
class Coche(private val marca: String, private val modelo: String) {
    companion object {
        var volanteALaDerecha: Boolean = true // Por defecto, para
países con conducción a la izquierda
```

```
        fun configurarConduccion(esConduccionALaDerecha: Boolean) {
            volanteALaDerecha = esConduccionALaDerecha
        }
    }

    fun describir() {
        val ladoVolante = if (volanteALaDerecha) "derecha" else
"izquierda"
        println("Soy un coche de la marca $marca y modelo $modelo.
Tengo el volante a la $ladoVolante.")
    }
}
```

Con este enfoque, puedes cambiar la configuración del volante para todos los coches según el país, utilizando el método `configurarConduccion` del `companion object`.

2.6 Ejemplos de clases y objetos propios del lenguaje

En Kotlin, todo es un objeto en el sentido de que podemos hacer uso de los métodos y atributos en cualquier del objeto. Por ejemplo, `Int` es una clase y `1` es un objeto de la clase `Int`.

Aqui tienes las clases predefinidas de Kotlin. Un ejemplo común y sencillo en muchos lenguajes de programación orientada a objetos es la clase `String`. Esta clase es utilizada para representar cadenas de texto y ya viene definida en la mayoría de los lenguajes de programación modernos, incluyendo Java, Python, C#, Kotlin, entre otros.

2.6.1. Clase: String

- **Clase `String`**: Esta clase es una plantilla predefinida en el lenguaje de programación para representar secuencias de caracteres (texto). Tiene varios métodos que permiten manipular o consultar la cadena de texto, como `length()`, `toUpperCase()`, `substring()`, etc.

2.6.2. Objeto/Instancia: Cadena de Texto

- **Objeto `String`**: Un objeto de la clase `String` es una instancia específica, es decir, una cadena de texto concreta. Por ejemplo, `"Hola Mundo"` es un objeto de la clase `String`.

2.6.3. Ejemplo en kotlin:

```
val saludo = String("Hola Mundo") // Creando un objeto de la clase
String. Tambien se puede usar "Hola Mundo".
val longitud = saludo.length // Utilizando una propiedad de la clase
String
println("Longitud del saludo: $longitud") // Mostrando la longitud de
la cadena
```

En este ejemplo:

- `saludo` es un objeto de la clase `String`.

- Utilizamos la propiedad `length` de la clase `String` para obtener la longitud de la cadena de texto.
- Imprimimos la longitud del objeto `saludo`.

Este ejemplo demuestra cómo se define un objeto `String` en Kotlin y cómo se puede interactuar con él utilizando las propiedades y métodos definidos en la clase `String`. La clase proporciona la estructura y los métodos para manipular cadenas de texto, mientras que el objeto `saludo` es una instancia específica con su propio valor (`"Hola Mundo"`).

3. Programa simple en programación orientada a objetos

La creación de programas es exactamente igual que hasta ahora. La diferencia es que ahora vamos a crear clases y objetos.

Vamos a desarrollar un programa simple en Kotlin que demuestra la programación orientada a objetos (POO). Este programa simulará un sistema de gestión comunitario donde:

- Se crean personas.
- Estas personas pueden adoptar animales.
- También pueden registrar sus vehículos.
- Finalmente, el programa mostrará un resumen de las personas con sus animales adoptados y vehículos registrados.

Utilizaremos clases para representar personas, animales y coches, demostrando conceptos clave de POO como la encapsulación, la instanciación de objetos y la interacción entre diferentes objetos.

Para desarrollar un programa simple en Kotlin haciendo uso de POO que haga lo que hemos comentado anteriormente, sigue estos pasos:

1. **Definir Clases**: Empieza explicando cómo se definen las clases en Kotlin. Cada clase (`Persona`, `Animal`, `Coche`) encapsula datos (atributos) y comportamientos (métodos).
2. **Crear Instancias**: Muestra cómo crear instancias de estas clases usando sus constructores. Por ejemplo, `val juan = Persona("Juan", 30)`.
3. **Usar Métodos y Propiedades**: Explica cómo acceder y utilizar métodos y propiedades de los objetos. Por ejemplo, `juan.presentarse()` llama al método `presentarse` del objeto `juan`.
4. **Estructuras de Datos**: Introduce el uso de estructuras de datos como listas y mapas para almacenar y organizar objetos.
5. **Iteración y Acción**: Muestra cómo iterar sobre estas estructuras (con `forEach`, por ejemplo) y realizar acciones, como imprimir información.
6. **Interacción entre Objetos**: Demuestra cómo diferentes objetos interactúan, como en el mapeo de personas con sus animales y vehículos.

Este enfoque proporciona una visión clara y estructurada de cómo se construyen y se utilizan objetos en un programa Kotlin, destacando los principios de la POO.

```
import com.sun.xml.internal.fastinfoset.util.PrefixArray

/**
 * Clase Persona representa a un individuo con un nombre y una edad.
 * @param nombre Nombre de la persona.
 * @param edad Edad de la persona.
 */
class Persona(private val nombre: String, private val edad: Int) {
   /**
    * Método para que la persona se presente.
    */
   fun presentarse() {
      println("Hola, mi nombre es $nombre y tengo $edad años.")
   }
}

/**
 * Clase Animal representa a un animal con una especie y un nombre.
 * @param especie Especie del animal.
 * @param nombre Nombre del animal.
 */
class Animal(private val especie: String, private val nombre: String)
{
   /**
    * Método para que el animal emita su sonido característico.
    */
   fun emitirSonido() {
      println("Soy un $especie y me llamo $nombre. ¡Hago un sonido
característico!")
   }
}

/**
 * Clase Coche representa un vehículo con marca y modelo.
 * @param marca Marca del coche.
 * @param modelo Modelo del coche.
 */
class Coche(private val marca: String, private val modelo: String) {
   /**
    * Método para describir el coche.
    */
   fun describir(prefijo:String  = "") {
      println(prefijo+"Soy un coche de la marca $marca y modelo
$modelo.")
   }
}

fun main() {
   // Registro de personas
   val juan = Persona("Juan", 30)
   val maria = Persona("Maria", 25)

   // Adopción de animales
   val perro = Animal("perro", "Firulais")
   val gato = Animal("gato", "Misu")
   val animalesAdoptados = mapOf(juan to perro, maria to gato)

   // Registro de vehículos
   val cocheJuan = Coche("Seat", "Ibiza")
   val cocheMaria = Coche("Ford", "Fiesta")
   val vehiculos = mapOf(juan to cocheJuan, maria to cocheMaria)
```

```
   // Mostrar información
   animalesAdoptados.forEach { (persona, animal) ->
      persona.presentarse()
      println("He adoptado un ${animal.especie} llamado
${animal.nombre}.")
      vehiculos[persona]?.describir("=>")
   }
}
```

4.3.2 KOTLIN Y PROGRAMACIÓN ORIENTADA A OBJETOS

Clases y objetos

Clases

Las clases son los bloques de construcción principales de cualquier lenguaje de programación orientado a objetos. Las clases son esencialmente **tipos personalizados**: un grupo de variables y métodos unidos en una estructura coherente. Para definir una clase se usa la palabra clave `'class'`.

```
class Invoice { ... }
```

La declaración de clase consiste en el nombre de la clase, el encabezado de la clase (especificando sus parámetros de tipo, el constructor primario, etc.) y el cuerpo de clase, rodeado de llaves. Tanto el encabezado como el cuerpo son opcionales. Si la clase no tiene cuerpo se pueden omitir las llaves.

Si no se especifica visibilidad, la visibilidad por defecto es `public` y por tanto cualquiera puede crear instancias de dicha clase.

```
class Empty
```

En comparación con Java, puede definir varias clases dentro del mismo archivo fuente.

La clases pueden contener:

- Constructores y bloques `'init'`
- Funciones
- Propiedades
- Clases anidadas e internas
- Declaraciones de tipo `'object'`

Constructores

Una clase en Kotlin puede tener un **constructor primario** y uno o más **constructores secundarios**.

El constructor primario es parte del encabezado de la clase. Este constructor va después del nombre de la clase (y los parámetros de tipo que son opcionales). Por defecto, todos los constructores son públicos, lo que equivale efectivamente a que sean visible en todas partes donde la clase sea visible.

```
class Person constructor(firstName: String) { ... }
```

Si el constructor principal no tiene anotaciones o modificadores de visibilidad, la palabra clave `'constructor'` se puede omitir:

```
// Podemos omitir la palabra clave 'constructor'
class Person(firstName: String) { ... }

// Las anotaciones o modificadores de visibilidad requieren la palabra
clave 'constructor'
class Customer public @Inject constructor(name: String) { ... }
```

Si una clase no-abstracta no declara ningún constructor (primario o secundario), tendrá un constructor primario sin argumentos generado automáticamente. La visibilidad del constructor será **pública** por defecto. Si no desea que su clase tenga un constructor público, es necesario declarar un constructor vacío con una visibilidad que no sea la predeterminada:

```
// Clase con un constructor privado
class DontCreateMe private constructor () { ... }
```

Para crear una instancia de una clase, se invoca al constructor como si de una función regular se tratase. En Kotlin no existe la palabra clave *'new'*:

```
class Person(val name: String) {
    constructor(name: String, parent: Person) : this(name) {
        parent.children.add(this)
    }
}

val person = Person("John")
```

Constructor primario

El **constructor primario** no puede contener ningún código. El código de inicialización se puede colocar en bloques de inicialización, que se definen con la palabra clave `'init'`.

Durante una inicialización de la instancia, los bloques de inicialización se ejecutan en el mismo orden en que aparecen en el cuerpo de la clase, intercalados con los inicializadores de propiedades:

```
class InitOrderDemo(name: String) {
    val firstProperty = "First property: $name"

    init {
        println("First initializer block that prints ${name}")
    }
```

```
    val secondProperty = "Second property: ${name.length}"

    init {
        println("Second initializer block that prints ${name.length}")
    }
}
```

Los bloques `'init'` pueden usarse para validar las propiedades o parámetros mediante la palabra clave `'require'`:

```
class Person (val firstName: String, val lastName: String, val age:
Int?) {
    init{
        require(firstName.trim().length > 0) { "Invalid firstName
argument." }
        require(lastName.trim().length > 0) { "Invalid lastName
argument." }

        if (age != null) {
            require(age >= 0 && age < 150) { "Invalid age argument." }
        }
    }
}
```

Tenga en cuenta que los parámetros del constructor primario se pueden usar en los bloques de inicialización. También pueden ser utilizados en los inicializadores de las propiedades en el cuerpo de la clase:

```
class Customer(name: String) {
    // Uso del parámetro 'name' para inicializar la propiedad
'customerKey'
    val customerKey = name.toUpperCase()
}
```

De hecho, para **declarar propiedades** e inicializarlas desde el constructor principal, Kotlin tiene una sintaxis concisa:

```
class Person(val firstName: String, val lastName: String, var age:
Int) { ... }
```

De la misma forma que las propiedades definidas en el cuerpo de la clase, las propiedades declaradas en el constructor primario pueden ser mutables (`'var'`) o de solo lectura (`'val'`).

Cuando se usa el prefijo `'val'` Kotlin genera automáticamente el método `'getter()'` y cuando se usa el prefijo `'var'` Kotlin genera el `'getter()'` y `'setter()'`. Si no necesitamos los accesores se puede definir el constructor sin los prefijos. De esta forma podemos definir nuestros propios métodos accesores.

En este ejemplo, el constructor principal de la primera clase define las propiedades, mientras que el segundo no lo hace:

```
// class with primary constructor that defines properties
class Info (var name: String, var number: Int)
```

```
// class with primary constructor that does not define properties
class Info (name: String, number: Int)
```

Constructor secundario

La clase también puede declarar uno o varios **constructores secundarios**, que se definen con la palabra clave `'constructor'`:

```
class Person {
    // Constructor secundario
    constructor(parent: Person) {
        parent.children.add(this)
    }
}
```

Si la clase tiene un constructor primario, cada **constructor secundario debe delegar en el constructor primario**, ya sea directamente o indirectamente a través de otro/s constructor/es secundario/s. La delegación en otro constructor de la misma clase se hace usando la palabra clave `'this'`:

```
class Person(val name: String) { // Constructor primario

    // Constructor secundario
    // Usamos 'this' para invocar al constructor primario
    constructor(name: String, parent: Person) : this(name) {
        parent.children.add(this)
    }
}
```

Hay que tenera en cuenta que el código en los bloques de inicialización se convierte efectivamente en parte del constructor primario. La delegación en el constructor primario ocurre como la primera instrucción en el constructor secundario, por lo que el código en todos los bloques de inicialización se ejecuta antes que el constructor secundario. Incluso si la clase no tiene un constructor primario, la delegación todavía ocurre implícitamente y los bloques de inicialización aún se ejecutan antes:

```
class Constructors {
    init {
        println("Init block") // Se ejecuta antes que el constructor
secundario
    }
    constructor(i: Int) {
        println("Constructor")
    }
}
```

La diferencia importante entre los constructores secundarios y primarios es que los parámetros de los constructores primarios pueden definir propiedades, mientras que los parámetros de un constructor secundario siempre son solo parámetros.

Si los parámetros de un constructor primario también son propiedades, serán accesibles a lo largo de todo el ciclo de vida del objeto, al igual que las propiedades normales. Mientras que, si son simples parámetros, obviamente sólo son accesibles dentro del constructor, como cualquier otro parámetro de una función.

Propiedades

En Kotlin no se utiliza el concepto de 'campo' cuando hablamos de variables de instancia sino que se emplea el concepto de **propiedades**.

Las propiedades de una clase pueden declararse como mutables *(`var`)*, o de inmutables o de sólo lectura *(`val`)*:

```
class Address {
    var name: String = ...
    var street: String = ...
    var city: String = ...
    var state: String? = ...
    var zip: String = ...
}
```

Para acceder a las propiedades de una clase usamos el operador punto `'.'` ya que a diferencia de Java no hay que utilizar `getters()` ni `setters()` si hemos definido la propiedad con `'val'` o `'var'`. Para usar la propiedad, simplemente nos referimos a ella por su nombre, como si fuera un campo en Java:

```
fun copyAddress(address: Address): Address {
    val result = Address() // there's no 'new' keyword in Kotlin
    result.name = address.name // accessors are called
    result.street = address.street
    // ...
    return result
}
```

'Getters()' and 'Setters()'

La sintaxis completa de definición de una propiedad en Kotlin:

```
{var|val} <propertyName>[: <PropertyType>] [= <property_initializer>]
    [<getter>]
    [<setter>]
```

El inicializador y las funciones `'getter()'` (y `'setter()'` si es una propiedad mutable) son opcionales. El tipo de la propiedad es opcional si puede inferirse desde el inicializador o desde el tipo de retorno del `'getter()'`.

```
var allByDefault: Int? // error: se requiere un inicializador
explícito.
var initialized = 1 // propiedad de tipo Int, getter y setter por
defecto

val simple: Int? // propiedad de tipo Int, getter por defecto, debe
ser inicializada por el constructor
val inferredType = 1 // propiedad de tipo Int y getter por defecto
```

Si las funciones `'getter()'` (y `'setter()'` en propiedades mutables) por defecto no son suficientes se puede codificar funciones `'getter()'` o `'setter()'` propias como cualquier otra función. Estas funciones están dentro de la propiedad y por tanto tienen que ser identadas correctamente

```
val isEmpty: Boolean
    get() = this.size == 0

var stringRepresentation: String
    get() = this.toString()
    set(value) {
        setDataFromString(value) // parses the string and assigns
values to other properties
    }
```

Nótese que por convención, el nombre del parámetro de la función `'setter()'` es `'value'` pero no es obligatorio y puede escogerse otro nombre.

Las propiedades pueden ser **'private'**, **'protected'**, o **'public'** (visibilidad por defecto).

Backing Fields

El campo de respaldo o ***'backing field'*** es un campo generado automáticamente para cualquier propiedad que solo puede usarse dentro de los accesores (getter o setter).

Estará presente solo si utiliza la implementación predeterminada de al menos uno de los accesores, o si un descriptor de acceso personalizado lo hace referencia a través del identificador `'field'`. Este campo de respaldo se usa para evitar la llamada recursiva y por tanto evitar un *'StackOverflowError'*.

Kotlin proporciona automáticamente este campo de respaldo. Se puede hacer referencia a este campo en los accesores utilizando el identificador `'field'`:

```
var counter = 0 // Note: the initializer assigns the backing field
directly
    set(value) {
        if (value >= 0) field = value
    }
```

Este campo es necesario ya que el siguiente código genera un *'StackOverflowError'*. Cuando Kotlin encuentra la propiedad 'selectedColor' llama al `'getter()'` correspondiente. Si usamos 'selectedColor' dentro de la definición del propio `'getter()'` es cuando se producen llamadas recursivas que acaban generando un desbordamiento de la pila. Kotlin provee del *'backing field'* para evitarlo.

```
var selectedColor: Int = someDefaultValue
        get() = selectedColor
        set(value) {
            this.selectedColor = value
            doSomething()
        }

// Código correcto
var selectedColor: Int = someDefaultValue
        get() = field
        set(value) {
            field = value
            doSomething()
        }
```

Constantes en tiempo de compilación

Las propiedades cuyo valor se conoce en el momento de la compilación se pueden marcar como constantes de tiempo de compilación utilizando el modificador `'const'`. Tales propiedades necesitan cumplir los siguientes requisitos:

- Top-level o miembros de un `'objet'`
- Inicializado con un valor de tipo String o un tipo primitivo
- No tener un `'getter()'` propio

Estas propiedades pueden ser utilizadas en anotaciones:

```
const val SUBSYSTEM_DEPRECATED: String = "This subsystem is
deprecated"

@Deprecated(SUBSYSTEM_DEPRECATED) fun foo() { ... }
```

Late-Initialized Properties and Variables

Normalmente, las propiedades declaradas con un tipo no nulo deben inicializarse en el constructor. Sin embargo, bastante a menudo esto no es conveniente. Por ejemplo, las propiedades se pueden inicializar mediante la inyección de dependencias, o en el método de configuración de una prueba de unidad. En este caso, no puede proporcionar un inicializador que no sea nulo en el constructor, pero aún así desea evitar las comprobaciones nulas al hacer referencia a la propiedad dentro del cuerpo de una clase.

Para manejar este caso, puede marcar la propiedad con el modificador `'lateinit'`:

```
public class MyTest {
    lateinit var subject: TestSubject

    @SetUp fun setup() {
        subject = TestSubject()
    }

    @Test fun test() {
        subject.method() // dereference directly
    }
}
```

Para usar este modificador hay que cumplir ciertos requisitos:

- Se puede usar únicamente en las propiedades `'var'` declaradas dentro del cuerpo de una clase. Por tanto no se puede usar en propiedades declaradas en el constructor principal.
- La propiedad no tiene un `'getter()'` o `'setter()'` personalizado.

Acceder a una propiedad antes de que haya sido inicializada lanzará una *'UninitializedPropertyAccessException'*.

Member Functions

Una función miembro es una función que se define dentro de una clase, objeto o interfaz. Las funciones miembro se invocan con el operador `'.'`:

```
class Sample() {
    fun foo() {
        print("Foo")
    }
}

Sample().foo() // crea una instancia de 'Sample' e invoca el método
'foo'
```

Herencia

La herencia es fundamental para la programación orientada a objetos. Nos permite crear nuevas clases que reutilizan, amplían y/o modifican el comportamiento de los preexistentes. La clase preexistente se llama **superclase** (o clase base), y la clase nueva que estamos creando se llama **clase derivada**. Una clase derivada obtendrá implícitamente todos los campos, propiedades y métodos de la superclase (y de la superclase de la superclase si es el caso).

Hay una restricción en cuanto a cuántas clases podemos heredar; en una JVM, solo puede tener **una clase base**. Pero se puede heredar de múltiples interfaces.

La herencia es transitiva. Si la clase C se deriva de la clase B y esa clase B se deriva de una clase A dada, entonces la clase C es una clase derivada de A.

Todas las clases en Kotlin tienen una superclase común `'Any'`, que es la superclase predeterminada para una clase sin supertipos declarados. Esta clase `'Any'` tiene unos pocos métodos básicos como `equals()` o `toString()`:

```
// Hereda de 'Any' implicitamente
class Example
```

Para declarar que una clase hereda de una clase base, colocamos el tipo de la clase base después de dos puntos en el encabezado de la clase derivada. Por defecto en Kotlin las clases están cerradas a la herencia, es decir, son `'final'`. Para permitir que una clase sea heredada, hay que utilizar la palabra clave `'open'`.

```
open class Base(p: Int)

// the derived class has a primary constructor
class DerivedWithConstructor(p: Int) : Base(p)
```

Si la clase derivada tiene un constructor primario, la clase base puede (y debe) inicializarse allí mismo, utilizando los parámetros del constructor primario.

Si la clase no tiene un constructor primario, entonces cada constructor secundario tiene que inicializar el tipo base usando la palabra clave `'super'`, o delegar a otro constructor

que haga eso. Tenga en cuenta que en este caso, diferentes constructores secundarios pueden llamar a diferentes constructores de la clase base:

```
open class Base(p: Int) {
    constructor(p: Int, q: Int): this(p)
}

class DerivedWithoutConstructor : Base {
    // calling the base constructor with super()
    constructor(p: Int) : super(p)
}
```

Sobreescritura de métodos

Kotlin requiere **anotaciones explícitas** para la sobreescritura de funciones miembro.

Para que una función pueda ser sobreescrita se utiliza la palabra clave `'open'` delante del nombre de la función. Dado que las clases son **finales** en Kotlin, sólo podemos utilizar la palabra clave `'open'` en funciones miembro de clases que también hayan sido definidas como `'open'`.

Para indicar que una función en la clase derivada sobreescribe una función de la clase padre se utiliza la palabra clave `'override'` delante del nombre de la función. De esta forma le indicamos al compilador que esta función sobreescribe una función de la clase padre y puede realizar las comprobaciones en tiempo de compilación.

Una función con la palabra clave `'override'` también es `'open'` por definición y puede ser sobreescrita por las subclases sucesivas. Es posible marcar una función `'override'` con la palabra clave `'final'` para evitar que sea sobreescrita.

```
open class Base {
    open fun v() { ... }
    open fun x(p: Int) { ... }
    fun nv() { ... }
}

class Derived: Base() {
    override fun v() { ... }

    final override fun x(p: Int) { ... } // Restringir la
sobreescritura
}
```

En Kotlin, la herencia está regulada por la siguiente regla: si una clase hereda varias implementaciones del mismo miembro de sus superclases inmediatas, debe invalidar este miembro y proporcionar su propia implementación. Para denotar el supertipo del cual se toma la implementación heredada, usamos la palaba clave `'super'` calificado por el nombre de supertipo entre paréntesis angulares, por ejemplo, `super<Base>`:

```
open class A {
    open fun f() { print("A") }
    fun a() { print("a") }
}

interface B {
```

```
    fun f() { print("B") } // interface members are 'open' by default
    fun b() { print("b") }
}

class C() : A(), B {
    // El compilador requiere que 'f()' sea sobreescrito para eliminar
la ambigüedad
    override fun f() {
        super<A>.f() // call to A.f()
        super<B>.f() // call to B.f()
    }
}
```

En la sobreescritura de métodos con valores por defecto siempre se utilizan los mismos valores de parámetros por defecto que el método base. Cuando se sobreescribe un método, los valores por defecto deben omitirse de la firma:

```
open class A {
    open fun foo(i: Int = 10) { ... }
}

class B : A() {
    override fun foo(i: Int) { ... }  // no default value allowed
}
```

Sobreescritura de propiedades

La sobreescritura de propiedades funciona de manera similar a la sobreescritura de métodos.

Las propiedades declaradas en una superclase que luego se vuelven a declarar en una clase derivada deben ir precedidas por la palabra clave `'override'` y deben tener un tipo compatible. También se puede usar la palabra clave `'override'` como parte de la declaración de una propiedad en un constructor primario.

Cada propiedad declarada puede ser sobreescrita por una propiedad con un inicializador o por una propiedad con un método `'getter()'`

```
open class Foo {
    open val x: Int get() { ... }
}

class Bar : Foo() {
    override val x: Int = ...
}

interface Foo1 {
    val count: Int
}

class Bar1(override val count: Int) : Foo1
```

Orden de inicialización

Durante la construcción de una nueva instancia de una clase derivada, **la inicialización de la clase base se realiza como primer paso** (precedida solo por la evaluación de los

argumentos para el constructor de la clase base) y, por lo tanto, ocurre antes de que se ejecute la lógica de inicialización de la clase derivada.

Por lo tanto, durante la inicialización de las propiedades de la clase base las propiedades de la clase derivada aún no se han inicializado. Si alguna de esas propiedades se utilizan (de forma directa o indirecta) en la inicialización de la clase base se pueden producir comportamientos extraños o errores en tiempo de ejecución.

```
open class Base(val name: String) {
    init {
        println("Initializing Base")
    }

    open val size: Int =
            name.length.also { println("Initializing size in Base:
$it") }
}

class Derived(name: String, val lastName: String) :
Base(name.capitalize().also { println("Argument for Base: $it") }) {
    init {
        println("Initializing Derived")
    }

    override val size: Int =
            (super.size + lastName.length).also {
println("Initializing size in Derived: $it") }
}

// Argument for Base: Hello
// Initializing Base
// Initializing size in Base: 5
// Initializing Derived
// Initializing size in Derived: 10
```

Invocar la implementación de la superclase

El código en una clase derivada puede llamar a funciones en la superclase e implementaciones de accesores de propiedades usando la palabra clave `'super'`:

```
open class Foo {
    open fun f() { println("Foo.f()") }
    open val x: Int get() = 1
}

class Bar : Foo() {
    override fun f() {
        super.f()  // Calling the super function
        println("Bar.f()")
    }
    override val x: Int get() = super.x + 1
}
```

Clases abstractas

Kotlin admite **clases abstractas** al igual que Java. Una clase abstracta es una clase con métodos marcados como abstractos y que por tanto no puede ser instanciada. Si una clase tiene uno o varios métodos abstractos es una clase abstracta y se indica con la palabra clave `'abstract'`.

La subclase concreta de una clase abstracta deberá implementar todos los métodos y propiedades definidos en la clase abstracta; de lo contrario, también será considerada como una clase abstracta.

```
open class Person {
    open fun fullName(): String { ... }
}

abstract class Employee (val firstName: String, val lastName: String):
Person() {
    // Variable de intancia en una clase abstracta
    val propFoo: String = "bla bla"

    abstract fun earnings(): Double

    // Podemos tener métodos con implementación por defecto
    override fun fullName(): String {
        return lastName + " " + firstName;
    }
}
```

Las clases abstractas pueden contener métodos con implementación por defecto como cualquier otra clase. Las subclases de la clase abstracta pueden sobreescribir la implementación predeterminada de un método pero solo si el método tiene el modificador `'open'`. Los métodos marcados como `'abstract'` también son `'open'` por defecto. Las clases abstractas también pueden definir variables de instancia al contrario que pasa con las interfaces.

Interfaces

Las interfaces en Kotlin son muy similares a Java 8. Pueden contener declaraciones de métodos abstractos, así como implementaciones de métodos. Lo que los diferencia de las clases abstractas es que las interfaces no pueden almacenar el estado, es decir, no pueden tener variables de instancia. Pueden tener propiedades, pero estas deben ser abstractas o proporcionar implementaciones de accesores.

Una interfaz se define usando la palabra clave `'interface'`. Un método en una interfaz es abstracto por defecto si no se proporciona una implementación.

```
interface MyInterface {
    fun bar()  // abstract by default
    fun foo() {
        // optional body
    }
}
```

Una clase u objeto pueden implementar una o varias interfaces:

```
class Child : MyInterface {
    override fun bar() {
        // body
    }
}
```

En una interfaz se pueden declarar propiedades. Una propiedad declarada en una interfaz puede ser abstracta o puede proporcionar implementaciones para el `'getter()'` o `'setter()'`. Las propiedades declaradas en interfaces no pueden tener *'backing fields'* y, por lo tanto, los accesores declarados en interfaces no pueden hacer referencia a ellos.

```
interface MyInterface {
    val prop: Int // abstract

    val propertyWithImplementation: String
        get() = "foo"

    fun foo() {
        print(prop)
    }
}

class Child : MyInterface {
    override val prop: Int = 29
}
```

Una interfaz puede derivar de otras interfaces y, por lo tanto, proporcionar implementaciones para sus miembros y declarar nuevas funciones y propiedades. Naturalmente, las clases que implementen dicha interfaz solo tienen que definir las implementaciones que faltan:

```
interface Named {
    val name: String
}

interface Person : Named {
    val firstName: String
    val lastName: String
    override val name: String get() = "$firstName $lastName"
}

data class Employee(
    // implementing 'name' is not required
    override val firstName: String,
    override val lastName: String,
    val position: Position
) : Person
```

En el caso de clases que hereden de varias interfaces, para evitar ambigüedades la subclase deberá proporcionar implementaciones tanto para métodos que tienen una implementación en una de las interfaces como en métodos que tiene implementaciones en varias interfaces.

```
interface A {
```

```
    fun foo() { print("A") }
    fun bar()  // abstract
}

interface B {
    fun foo() { print("B") }
    fun bar() { print("bar") }
}

class C : A {
    override fun bar() { print("bar") }
}

// la clase 'D' tieen que implementar tanto foo() como bar()
class D : A, B {
    override fun foo() {
        super<A>.foo()
        super<B>.foo()
    }

    override fun bar() {
        super<B>.bar()
    }
}
```

Visibilidad

Las clases, objetos, interfaces, constructores, funciones, propiedades y sus *'setters'* pueden tener modificadores de visibilidad. (Los *'setters'* siempre tienen la misma visibilidad que la propiedad).

- **Public** - Este es el valor predeterminado, y se puede acceder a cualquier clase, función, propiedad, interfaz u objeto que tenga este modificador desde cualquier lugar.
- **Private** - Se puede acceder a una función, interfaz o clase de nivel superior que se declara como privada solo dentro del mismo archivo.

Cualquier función o propiedad que se declare privada dentro de una clase, objeto o interfaz solo puede ser visible para otros miembros de esa misma clase, objeto o interfaz.

Un constructor privado debe usar la palabra clave `'constructor'`. Si un constructor es marcado como privado no se puede instanciar un objeto con ese constructor.

```
class Car private constructor(val name: String, val plateNo: String) {
    // ....
}
```

- **Protected** - Solo se puede aplicar a propiedades o funciones dentro de una clase, objeto o interfaz, no se puede aplicar a funciones, clases o interfaces de nivel superior. Las propiedades o funciones con este modificador solo son accesibles dentro de la clase que lo define y cualquier subclase.
- **Internal** - En un proyecto que tiene un módulo (módulo Gradle o Maven), una clase, objeto, interfaz o función especificada con este modificador dentro de ese módulo solo es accesible desde ese módulo.

Data classes

Las **Data classes** son una forma concisa de crear clases que solo contienen datos. Estas clases se definen con la palabra clave `'data'`.

```
data class User(val name: String, val age: Int)
```

De forma automática el compilador crear los métodos `hashCode()`, `equals()`, `copy()` y `toString()` a partir de todas las propiedades declaradas en el constructor primario. También se generan las funciones `componentN()` que corresponden a las propiedades declaradas en orden en el constructor primario.

Para evitar comportamientos extraños estas clases deben **cumplir ciertos requisitos**:

- El constructor primario necesita tener al menos un parámetro.
- Todos los parámetros del constructor primario estarán marcados como `'val'` o `'var'`.
- Una *'data class'* no puede ser `'abstract'`, `'open'`, `'sealed'` o `'inner'`.
- (Antes de 1.1) Las *'data classes'* no pueden extender de otras clases (pero pueden implementar interfaces).

El compilador sólo tiene en cuenta las propiedades declaradas en el constructor primario a la hora de generar los métodos de forma automática. Por tanto, para excluir propiedades se deben declarar en el cuerpo de la clase.

```
data class DataClassExample(val x: Int, val y: Int, val z: Int) {
    // Propiedad excluida
    var xx; Int = 0
}

val fooData = DataClassExample(1, 2, 4)
val fooCopy = fooData.copy(y = 100)

// El formato de 'toString()' es el mismo 'ClassName(prop=xx, prop=yy,
....)'
println(fooData) // => DataClassExample(x=1, y=2, z=4)
println(fooCopy) // => DataClassExample(x=1, y=100, z=4)
```

El compilador genera la función `copy()` que permite copiar un objeto y en caso necesario, crear la copia alterando algunas de sus propiedades y manteniendo el resto.

```
data class User(val name: String, val age: Int)

// Función 'copy()' generada automáticamente
// fun copy(name: String = this.name, age: Int = this.age) =
User(name, age)

val jack = User(name = "Jack", age = 1)

// Copiamos el objeto pero modificando la propiedad 'age'
val olderJack = jack.copy(age = 2)
```

Las funciones `componentN()` permite desestructurar las propiedades:

```
val jane = User("Jane", 35)
val (name, age) = jane
println("$name, $age years of age") // => Jane, 35 years of age
```

Cada tipo se deriva de `'Any'`, que viene con una declaración de método `'hashCode()'`. Esto es el equivalente de un método `'hashCode()'` de clase *'Object'* de Java. Este método es importante cuando se insertan instancias del objeto en colecciones, como un mapa. Al implementar este método, se debe cumplir con una serie de requisitos:

1. Cuando se invoque en el mismo objeto más de una vez durante el tiempo de ejecución, el método `'hashCode()'` debe devolver constantemente el mismo valor, dado que el objeto no se modificó.
2. Si para dos objetos el método `'equals()'` devuelve true, entonces llamar al método `'hashCode()'` en cada uno de ellos debería devolver el mismo valor entero.
3. Si dos objetos no son iguales, es decir, que el método `'equals()'` devuelve false cuando se comparan, no es un requisito que cada método `'hashCode()'` del objeto devuelva valores distintos. Sin embargo, producir un entero distinto para objetos desiguales podría mejorar el rendimiento de las colecciones basadas en 'hash'.

Las *'data classes'* son un forma compacta y legible de devolver dos o más valores de una función. Otra alternativa, menos legible, es utilizar el tipo `'Pair'` o `'Triple'` proporcionado por Kotlin:

```
data class Result(val result: Int, val status: Boolean)

fun checkStatus() = Result(10, true)  // función que retorna un tipo
'Result'

val (result, status) = checkStatus() // usamos la desestructuración de
datos para acceder a los datos
```

Sealed classes

En Kotlin una ***'sealed class'*** es una clase abstracta (no se puede crear instancias) que otras clases pueden extender. Estas subclases se definen dentro del cuerpo de la *'sealed class'*, en el mismo archivo por lo que podemos conocer todas las subclases posibles simplemente viendo el archivo.

Las *'sealed class'* se utilizan para representar jerarquías de clases restringidas, de forma que una clase solo pueda heredar de un conjunto limidado de tipos. Son, en cierto sentido, una extensión de las clases de enumeración.

- Podemos agregar el modificador `'abstract'`, pero esto es redundante porque estas clases son abstractas por defecto.
- No pueden tener el modificador `'open'` ni `'final'`.
- Podemos declarar clases de datos y objetos como subclases a una *'sealed class'* (aún deben declararse en el mismo archivo).

- No pueden tener constructores públicos ya que sus constructores son privados de forma predeterminada.

```
// shape.kt

sealed class Shape

class Circle : Shape()
class Triangle : Shape()
class Rectangle: Shape()
```

Generics

'Covariance' y ***'contravariance'*** son términos que hacen referencia a la capacidad de usar un tipo más derivado (más específico) o menos derivado (menos específico) que el indicado originalmente. Los parámetros de tipo genérico admiten estos términos para proporcionar mayor flexibilidad a la hora de asignar y usar tipos genéricos. Cuando se hace referencia a un sistema de tipos, se definen como:

- ***'Covariance'*** -> Permite usar un tipo más derivado que el especificado originalmente. Puede asignar una instancia de `Class<Derived>` a una variable de tipo `Class<Base>`.
- ***'Contravariance'*** -> Permite usar un tipo más genérico (menos derivado) que el especificado originalmente. Puede asignar una instancia de `Class<Base>` a una variable de tipo `Class<Derived>`.
- ***'Invariance'*** -> Significa que solo se puede usar el tipo especificado originalmente. Así, un parámetro de tipo genérico invariable no es covariante ni contravariante. No se puede asignar una instancia de `List<Base>` a una variable de tipo `List<Derived>` o viceversa.

Al igual que en Java, en Kotlin las clases pueden tener tipos con parámetros.

```
class Box<T>(t: T) {
    var value = t
}
```

En general, para crear una instancia de una clase genérica tenemos que proveer el tipo a la clase:

```
val box: Box<Int> = Box<Int>(1)
```

Si los parámetros se pueden inferir, como por ejemplo de los argumentos del constructor o por algún otro medio, se pueden omitir los argumentos de tipo:

```
val box = Box(1) // '1' tiene tipo Int así que el compilador infiere
el tipo "Box<Int>"
```

La palabra clave 'out'

Digamos que queremos crear una clase de productor que producirá un resultado de algún tipo 'T'. A veces; queremos asignar ese valor producido a una referencia que es de un supertipo del tipo 'T'.

Para lograr eso usando Kotlin, necesitamos usar la palabra clave `'out'` en el tipo genérico. Esto significa que podemos asignar esta referencia a cualquiera de sus supertipos. El valor de salida solo puede ser producido por la clase dada pero no consumido:

```
class ParameterizedProducer<out T>(private val value: T) {
    fun get(): T {
        return value
    }
}

val a = ParameterizedProducer("string") //
ParameterizedProducer<String>
val x: ParameterizedProducer<Any> = a // Correcto

val b = ParameterizedProducer(10) // ParameterizedProducer<Int>
val y: ParameterizedProducer<Number> = b // Correcto
val z: ParameterizedProducer<String> = b // ¡Error de compilación!
```

La palabra clave 'in'

A veces, tenemos una situación opuesta, lo que significa que tenemos una referencia de tipo T y queremos poder asignarla al subtipo de T.

Podemos usar la palabra clave `'in'` en el tipo genérico si queremos asignarlo a la referencia de su subtipo. La palabra clave `'in'` solo se puede utilizar en el tipo de parámetro que se consume, no se produce:

```
class ParameterizedConsumer<in T> {
    fun toString(value: T): String {  // 'toString()' will only be
consuming a value of type T.
        return value.toString()
    }
}

val a = ParameterizedConsumer<Number>()

val b: ParameterizedConsumer<Double> = a // Correcto
val c: ParameterizedConsumer<Int> = a // Correcto
val d: ParameterizedConsumer<String> = a // ¡Error de compilación!
```

Star projections

Hay situaciones en las que no es importante el tipo específico de un valor. Para ello usamos el operador `'*'` o *'star projection'*:

```
fun printArray(array: Array<*>) {
    array.forEach { println(it) }
}

// Podemos pasar una matriz de cualquier tipo al método 'printArray()'
printArray(arrayOf(1,2,3))

printArray(arrayOf("hello", "World!!", 5))
```

Generic functions

Las funciones también pueden ser genéricas en los tipos que utilizan. Esto permite escribir una función que puede funcionar con cualquier tipo, en lugar de solo un tipo específico. Para ello, definimos los parámetros de tipo en la firma de función.

```
fun <T> choose(t1: T, t2: T, t3: T): T {
    return when (Random().nextInt(3)) {
        0 -> t1
        1 -> t2
        else -> t3
    }
}

// Podemos usar esta función con enteros. Si el compilador puede
inferir el tipo se puede omitir.
val r = choose<Int>(5, 7, 9)
val r = choose(5, 7, 9)

// También es válido usar la función con Strings
val s = choose<String>("BMW", "Audi", "Ford")
val s = choose("BMW", "Audi", "Ford")
```

Generic constraints

El conjunto de todos los tipos posibles que pueden sustituirse por un parámetro de tipo dado puede estar restringido por restricciones genéricas.

El tipo más común de restricción es un límite superior que corresponde a la palabra clave de extensión de Java:

```
fun <T : Comparable<T>> sort(list: List<T>) {  ... }

sort(listOf(1, 2, 3)) // OK. Int is a subtype of Comparable<Int>
sort(listOf(HashMap<Int, String>())) // Error: HashMap<Int, String> is
not a subtype of Comparable<HashMap<Int, String>>
```

El límite superior predeterminado (si no se especifica) es `'Any?'`.

Nested classes

Al igual que las funciones, Kotlin permite las clases internas, es decir, clases definidas dentro de otra clase. Son equivalentes a las clases internas estáticas en Java.

```
class OuterClass {

    class NestedClass {
        fun nestedClassFunc() { }
    }
}

val nestedClass = OuterClass.NestedClass().nestedClassFunc()
```

Inner class

Las clases internas, por otro lado, pueden hacer referencia a la clase externa en la que se declaró. Para crear una clase interna, colocamos la palabra clave `'inner'` antes de la palabra clave `'class'`.

```
class OuterClass() {
    val oCPropt: String = "Yo"

    inner class InnerClass {
        fun innerClassFunc() {
            val outerClass = this@OuterClass
            print(outerClass.oCPropt)
        }
    }
}
val demo = OuterClass().InnerClass().innerClassFunc() // => yo
```

Enumeraciones

Las clases de enumeración son similares a los tipos *'enum'* de Java. El uso más básico de las clases de enumeración es la implementación de enumeraciones de tipos seguros. Cada constante de la enumeración es un objeto. Las constantes de la enumeración están separadas por comas.

```
enum class Country {
    Spain, France, Portugal
}
```

Las enumeraciones pueden tener constructor:

```
enum class Direction(val angle: Int) {
    North(90), West(180), South(270), East(0)
}
```

En Kotlin las constantes de la enumeración pueden declarar sus propias clases anónimas con sus métodos correspondientes, así como sobreescribir métodos primarios.

Si la enumeración define algún miembro, debe separar las definiciones de constantes de enumeración de las definiciones de miembros con un punto y coma, al igual que en Java.

```
enum class ProtocolState {
    WAITING {
        override fun signal() = TALKING
    },

    TALKING {
        override fun signal() = WAITING
    };

    abstract fun signal(): ProtocolState
}
```

En Kotlin las enumeraciones disponen de forma predeterminada de los métodos:

- `EnumClass.valueOf(value: String): EnumClass` -> Devuelve la constante de enumeración por su nombre. Lanza un *'IllegalArgumentException'* si no existe la constante.
- `EnumClass.values(): Array<EnumClass>` -> Retorna un array con las constantes de enumeración.

Además de los métodos las instancias de enumeración vienen con dos propiedades predefinidas. Uno es `'name'` de tipo 'String' y el segundo es `'ordinal'` de tipo 'Int' para obtener la posición de la constante dentro de la enumeración, teniendo en cuenta que empiezan por 0:

```
enum class Country {
    Spain, France, Portugal
}

println(Country.Spain) // => Spain
println(Country.valueOf("Spain")) // => Spain

println(Country.Portugal.name) // => Portugal
println(Country.France.ordinal) // => 1

fun countries() {
    for (country in Country.values()) {
        println("Country: $country")
    }
}
```

Objects

Los objetos son muy similares a las clases. A veces necesitamos crear un objeto con una ligera modificación de alguna clase, sin declarar explícitamente una nueva subclase para ello. Java maneja este caso con clases internas anónimas. Kotlin generaliza ligeramente este concepto con *'object expressions'* y *'objects declarations'*.

Estas son algunas de las características de los objetos en Kotlin:

- Pueden tener propiedades, métodos y un bloque init.
- Estas propiedades o métodos pueden tener modificadores de visibilidad.
- No pueden tener constructores (primarios o secundarios).
- Pueden extender otras clases o implementar una interfaz.

Hay importantes diferencias semánticas entre un *'object expression'* y un *'object declaration'*

- Los *'object expression'* se ejecutan (y se inicializan) inmediatamente, donde se usan.
- Los *'object declaration'* se inicializan cuando se accede por primera vez.
- Por su parte, un *'companion object'* se inicializa cuando se cargala clase correspondiente.

Objects expressions

Para crear un objeto de una clase anónima que hereda de algún tipo (o tipos), escribimos:

```
fun countClicks(window: JComponent) {
    var clickCount = 0
    var enterCount = 0

    window.addMouseListener(object : MouseAdapter() {
        override fun mouseClicked(e: MouseEvent) {
            clickCount++
        }

        override fun mouseEntered(e: MouseEvent) {
            enterCount++
        }
    })
    // ...
}
```

Objects declarations

Colocamos la palabra clave `'object'` antes del nombre del objeto que queremos crear. De hecho, estamos creando un **SINGLETON** cuando creamos objetos en Kotlin usando esta construcción ya que solo existe una instancia de un objeto.

```
object ObjectExample {
    val baseUrl: String = "http://www.myapi.com/"
    fun hello(): String {
        return "Hello"
    }
}

println(ObjectExample.hello()) // => Hello

fun useObject() {
    ObjectExample.hello() // => Hello
    val someRef: Any = ObjectExample // Usamos el nombre de los
objetos tal como son
}
```

Al igual que una declaración de variable, una declaración de objeto no es una expresión y no se puede utilizar en el lado derecho de una declaración de asignación.

Los objetos en Kotlin pueden utilizarse también para crear constantes.

```
object APIConstants {
    val baseUrl: String = "http://www.myapi.com/"
}
```

Companion objects

Los *'companion objects'* son un tipo de *'object declaration'*. Como Kotlin no admite clases, métodos o propiedades estáticas como las que tenemos en Java, Kotlin provee los *'companion objects'*. Estos objetos son básicamente un objeto que pertenece a una

clase que se conoce como la clase complementaria del objeto. Este objeto se indica con la palabra clave `'companion'`.

Similar a los métodos estáticos en Java, un *'companion object'* no está asociado con una instancia de clase, sino con la propia clase.

Se puede llamar a los miembros del *'companion object'* usando simplemente el nombre de la clase como el calificador, como si fuera un método estático.

Un *'companion object'* puede tener nombre que facilitará el ser invocado desde Java aunque es opcional.

```
class Person private constructor(var firstName: String, var lastName:
String) {

    // Podemos omitir el nombre del objeto
    companion object {
        var count: Int = 0
        fun create(firstName: String, lastName: String): Person =
Person(firstName, lastName)

        // Podemos tener bloques 'init' dentro de un 'companion
object'
        init {
            println("Person companion object created")
        }
    }
}
val person = Person.create("John", "Doe")

class MyClass {

    fun sayHello() = println("hello")

    // Objeto con el nombre 'Factory' y que utilizaremos como 'Factory
Pattern'
    companion object Factory {
        fun create(): MyClass = MyClass()

        fun sayHelloFromCompanion() = MyClass().sayHello() // Podemos
acceder a miembros de la clase
    }
}

val myClass = MyClass.create()
MyClass().sayHello() // incorrecto
MyClass.Factory.sayHelloFromCompanion() // Invocar un método del
'companion'
```